

Cloud project

on

**“AI-BASED RESUME PARSING AND JOB
MATCHING SYSTEM”**

Submitted by :

Aatif Sayyed

Ojaswi Mohapatra

Mohammad Reefath

Abdulharis Shaikh

1. Introduction

The recruitment process involves manually scanning hundreds of resumes to shortlist potential candidates. This is a time-consuming, inefficient, and potentially biased process. The recruiter needs to manually scan each resume, extract relevant skills, assess experience, and compare the candidate's skills to the job requirements. As the number of applications increases, manual screening becomes a bottleneck in the recruitment process.

To overcome these issues, this project proposes a cloud Resume Parser and Skill Matcher System using AWS serverless technologies. The system will automate the resume screening process and deliver structured and ranked results via a recruiter dashboard.

The system will automatically:

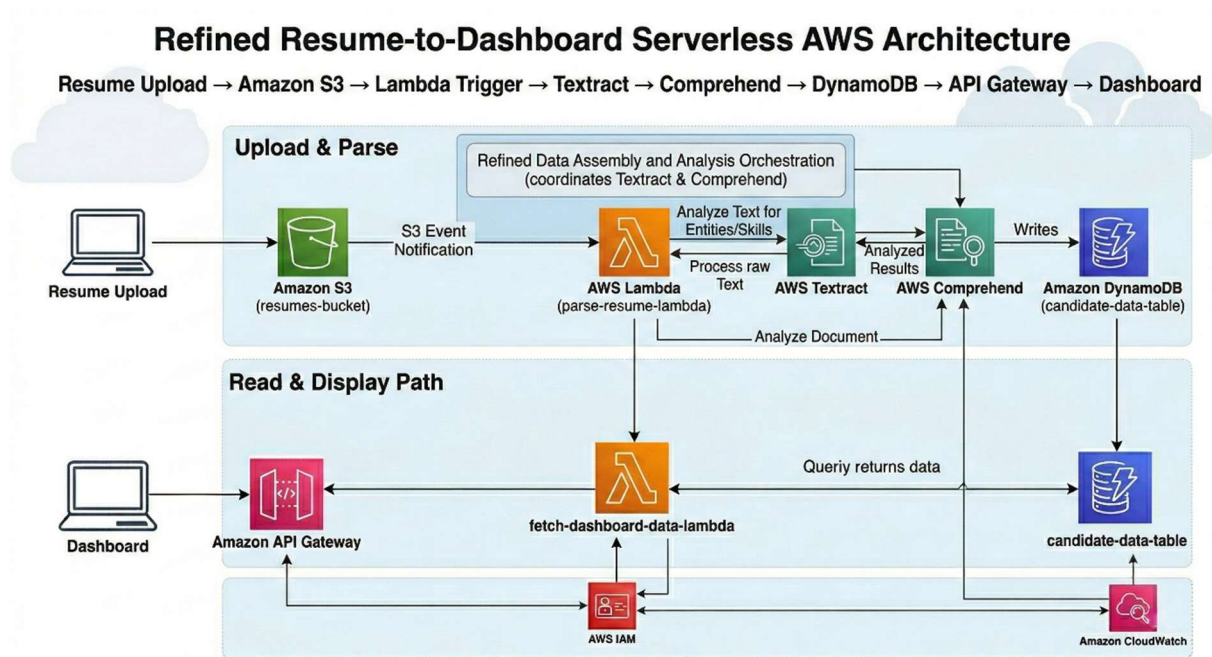
- Extract text from resumes in PDF format.
- Identify key information such as skills, experience, and education.
- Match candidates to job descriptions based on similarity algorithms (Jaccard similarity).
- Assign match scores to rank candidates objectively.
- Store structured candidate information in a cloud database.
- Present results via a REST API-based recruiter dashboard

The system will make the resume screening and candidate matching process more efficient, ensure consistent evaluation, and offer a scalable and secure recruitment solution.

2.Objectives

- The primary goals of this project are:
Automate Resume Text Extraction -Automatically extract text from PDF resumes using cloud document processing services.
- Extract Key Candidate Information:
Extract key information like skills, education, and experience using NLP and rule-based approaches.
- Implement Job Matching Mechanism:
Match candidates with job requirements using similarity scoring algorithms like Jaccard similarity to determine objective match percentages.
- Develop a Recruiter Dashboard:
Create a recruiter-friendly dashboard to view ranked candidates, filter results, and analyze match scores.
- Ensure Scalability and Security:
Develop the system using AWS serverless architecture to ensure high scalability, secure access control, and optimal performance.

3. System Architecture



Resume Upload → Amazon S3 → Lambda → Textract → Comprehend → DynamoDB → API Gateway → Dashboard

There are two main paths in the application:

- 1) Upload & Parse Path
- 2) Read & Display Path

1) Upload & Parse Path (Top Section)

This section handles resume processing.

Step 1: Resume Upload, Recruiter uploads PDF from laptop, File is uploaded to Amazon S3 (resumes-bucket).

Step 2: S3 Event Notification S3 automatically triggers a Lambda function, Lambda function name: parse-resume-lambda.

Step 3: AWS Lambda (Processing Engine), Lambda is the orchestrator: sends document to Amazon textract Sends extracted text to Amazon Comprehend Processes raw data.

Step 4: Amazon Textract Extracts raw text from PDF. converts document into machine-readable text.

Step 5: Amazon Comprehend analyzes text, Detects entities, keywords, skills returns structured results.

Step 6: DynamoDB processed candidate data is written to: candidate-data-table Stores skills, experience, match score, etc. So top section = Data Processing Pipeline.

2) Read & Display Path (Bottom Section)

This section deals with the display of the dashboard.

Step 7: Dashboard recruiter accesses web dashboard (HTML/JS frontend).

Step 8: API Gateway dashboard makes request to: Amazon API Gateway API Gateway provides safe REST APIs.

Step 9: Fetch Lambda API Gateway invokes: *“fetch-dashboard-data-lambda”*

This Lambda function: Accesses DynamoDB extracts candidate information

Step 10: Return Data is relayed from Lambda function to API Gateway API Gateway responds with JSON data to Dashboard shows ranked candidates.

4. Resume Extraction Process

Step 1: Resume Upload

Resumes are uploaded in PDF format to Amazon S3.

Step 2: Trigger Lambda

An S3 event trigger automatically invokes the ResumeParser Lambda function.

Step 3: Text Extraction

Amazon Textract extracts text from the PDF document.

Example extracted text:

"AWS Certified Developer with 5 years experience in Docker and Kubernetes..."

4.1. Skill Parsing Logic

The system extracts skills using:

1. Amazon Comprehend (Key phrase detection)
2. Custom skill matching logic (predefined skill list)

Example skill list:

- AWS
- Docker
- Kubernetes
- Python
- Java
- C
- React
- SQL

The extracted skills are stored in DynamoDB.

4.2. Experience Extraction

Experience is extracted using regular expression matching.

Example pattern:

`(\d+)\s+years`

If resume contains:

"6 years of experience"

The system extracts:

Experience = 6

4.3. Education Extraction

The system detects education keywords such as:

- B.E
- B.Tech
- Bachelor
- Master
- M.Tech
- MBA
- BCA
- MCA

This information is stored in DynamoDB under the education field.

4.4. Job Matching Logic

Job descriptions are stored in DynamoDB (Jobss table).

Example job:

job_id: cloud001

skills: AWS, Docker, Kubernetes, Python

The system uses Jaccard Similarity:

Match Score = $(\text{Intersection} / \text{Union}) \times 100$

Example:

Resume Skills: AWS, Docker, Python

Job Skills: AWS, Docker, Kubernetes, Python

Intersection = 3

Union = 4

Match Score = 75%

4.5. Data Storage (DynamoDB)

Two tables are used:

Candidatess Table

Stores:

- candidate_id
- resume_name
- skills
- experience
- education
- job_id
- match_score

Jobss Table

Stores:

- job_id
- skills

5. Recruiter Dashboard

The recruiter dashboard is built using HTML, CSS, and JavaScript. The dashboard is used for viewing the matched candidates.

➤ Key Features:

- Dark-themed professional user interface
- Job filter dropdown for selecting specific roles
- Match score color highlighting for easy comparison
- CSV report download option
- Automatically sorted results (highest match score first)

➤ Example Display Format:

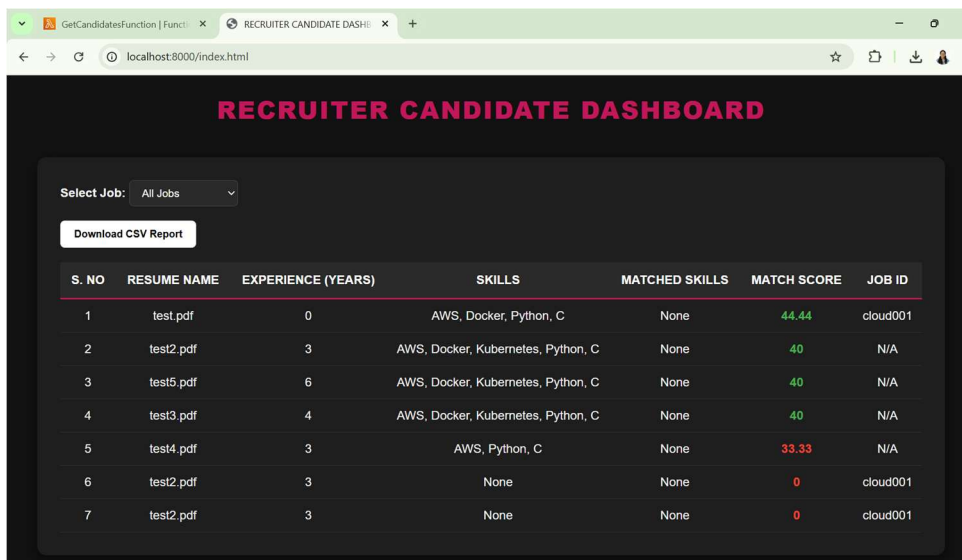
Resume Name | Experience | Skills | Match Score | Job ID

The dashboard fetches data from API Gateway and displays the ranked candidate results for recruiters.

6. Sample Output

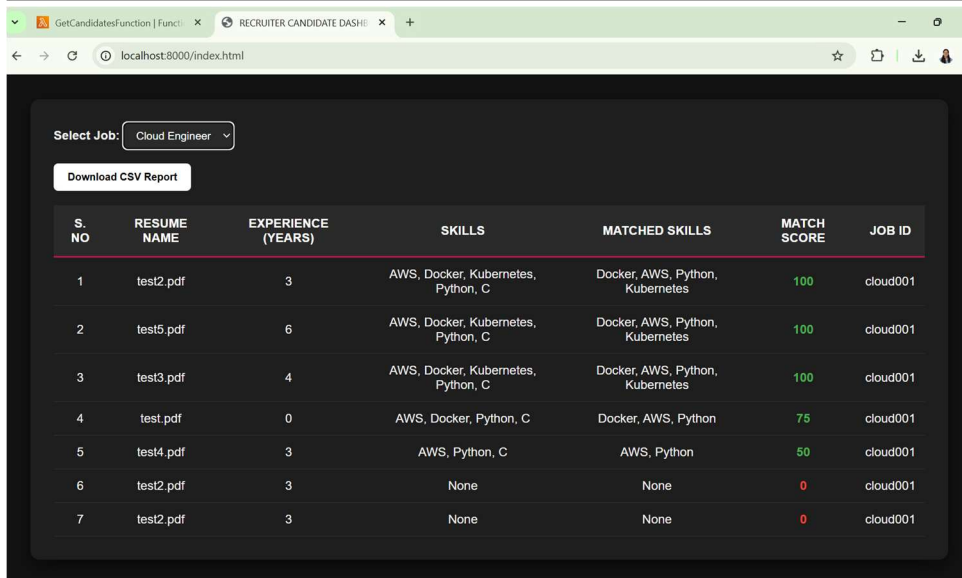
Example candidate record:

```
{
  "resume_name": "test5.pdf",
  "experience": 6,
  "skills": ["AWS", "Docker", "Kubernetes", "Python"],
  "education": ["BE"],
  "job_id": "cloud001",
  "match_score": 80
}
```



The screenshot shows a web browser window with the URL `localhost:8000/index.html`. The page title is "RECRUITER CANDIDATE DASHBOARD". Below the title, there is a "Select Job:" dropdown menu set to "All Jobs" and a "Download CSV Report" button. The main content is a table with 7 columns: S. NO, RESUME NAME, EXPERIENCE (YEARS), SKILLS, MATCHED SKILLS, MATCH SCORE, and JOB ID. The table contains 7 rows of candidate data.

S. NO	RESUME NAME	EXPERIENCE (YEARS)	SKILLS	MATCHED SKILLS	MATCH SCORE	JOB ID
1	test1.pdf	0	AWS, Docker, Python, C	None	44.44	cloud001
2	test2.pdf	3	AWS, Docker, Kubernetes, Python, C	None	40	N/A
3	test5.pdf	6	AWS, Docker, Kubernetes, Python, C	None	40	N/A
4	test3.pdf	4	AWS, Docker, Kubernetes, Python, C	None	40	N/A
5	test4.pdf	3	AWS, Python, C	None	33.33	N/A
6	test2.pdf	3	None	None	0	cloud001
7	test2.pdf	3	None	None	0	cloud001



The screenshot shows the same web browser window, but the "Select Job:" dropdown menu is now set to "Cloud Engineer". The "Download CSV Report" button is still present. The table now displays 7 rows of candidate data filtered for the "Cloud Engineer" job.

S. NO	RESUME NAME	EXPERIENCE (YEARS)	SKILLS	MATCHED SKILLS	MATCH SCORE	JOB ID
1	test2.pdf	3	AWS, Docker, Kubernetes, Python, C	Docker, AWS, Python, Kubernetes	100	cloud001
2	test5.pdf	6	AWS, Docker, Kubernetes, Python, C	Docker, AWS, Python, Kubernetes	100	cloud001
3	test3.pdf	4	AWS, Docker, Kubernetes, Python, C	Docker, AWS, Python, Kubernetes	100	cloud001
4	test1.pdf	0	AWS, Docker, Python, C	Docker, AWS, Python	75	cloud001
5	test4.pdf	3	AWS, Python, C	AWS, Python	50	cloud001
6	test2.pdf	3	None	None	0	cloud001
7	test2.pdf	3	None	None	0	cloud001

7. Implementation Status

The AI-Based Resume Parsing and Job Matching System has been successfully implemented end-to-end. The system is fully functional and demonstrates a complete automated recruitment workflow.

The following components are fully operational:

- Resume upload to Amazon S3
- Automatic Lambda trigger on file upload
- Text extraction using Amazon Textract
- Skill extraction using custom NLP logic
- Experience extraction using regex-based detection
- Education keyword detection
- Job descriptions stored in DynamoDB
- Jaccard similarity-based job matching
- Match score calculation
- Candidate data storage in DynamoDB
- REST API built using API Gateway
- Recruiter dashboard with filtering
- Match score sorting (highest first)
- CSV report download feature
- IAM-based access control
- CORS configuration for frontend access

The system works in real time and provides structured, ranked candidate data to recruiters.

8. Minor Limitations

Although the system is fully functional, a few minor improvements can enhance scalability and intelligence:

1. Skill Extraction is Keyword-Based
Currently, skill detection relies on predefined keywords.
A trained NLP model could further improve synonym recognition.
2. Matching Algorithm
Jaccard similarity is implemented successfully.
Cosine similarity with TF-IDF could provide slightly more precise ranking.
3. Education Detection
Education extraction is rule-based.
Structured entity extraction could improve formatting consistency.
4. High-Volume Upload Handling
Currently, S3 triggers Lambda directly.
For extremely large-scale enterprise systems, Amazon SQS could be introduced to queue requests and improve large-batch scalability

9.Future Enhancements

Although the system is fully functional, the following improvements can further enhance its intelligence, scalability, and production readiness.

1. Advanced Skill & Semantic Matching

Currently, the system uses keyword-based skill extraction and Jaccard similarity for matching.

Enhancement:

- Implement semantic matching using NLP embeddings.
- Use TF-IDF with cosine similarity for better ranking accuracy.
- Detect related skills (e.g., “AWS EC2” mapped to “AWS”).

This would significantly improve matching precision and reduce dependency on exact keyword matches.

2. Multi-Job Ranking System

Currently, resumes are matched against a selected job role.

Enhancement:

- Automatically match each resume against all available job descriptions.
- Store and display the top 3 matching jobs per candidate.
- Provide job recommendation ranking.

This transforms the system from a single-role matcher to a recommendation engine.

3. Scalable Event-Driven Architecture (SQS Integration)

Currently, S3 directly triggers Lambda.

Enhancement:

- Introduce Amazon SQS between S3 and Lambda.
- Queue resume processing requests.
- Enable batch processing and failure retries.

This ensures better scalability and stability for high-volume enterprise use.

4. Authentication & Role-Based Access

Currently, the dashboard is open-access.

Enhancement:

- Integrate Amazon Cognito for user authentication.
- Add recruiter/admin roles.
- Restrict API access using secured endpoints.

This would make the system production-ready and enterprise-grade.